

《数据安全》实验报告

零知识证明实践

姓名：林晖鹏 学号：2312966 专业：计算机科学与技术

目录

1 实验要求	3
2 实验原理	3
2.1 实验背景	3
2.2 零知识证明的基本性质	3
2.3 从承诺开始理解证明原理	3
2.4 非交互化处理	4
2.5 进阶实验原理：基于 libsnark 的通用零知识证明	5
2.5.1 约束系统的构造	5
2.5.2 R1CS 与 zkSNARK 流程	5
3 实验过程	6
3.1 实验环境	6
3.2 实验参数	6
3.3 实验模拟步骤	6
3.3.1 步骤一：验证公共方程并确定 witness	6
3.3.2 步骤二：生成公开陈述	7
3.3.3 步骤三：模拟承诺、挑战与响应	7
3.3.4 步骤四：验证证明	7
3.4 核心代码说明	7
3.5 程序运行结果	9
3.6 进阶实验：libsnark 实现模拟过程	10
3.6.1 步骤一：构造 R1CS 约束系统	10
3.6.2 步骤二：设置公开输入与私密 witness	10
3.6.3 步骤三：生成证明密钥、验证密钥与证明	10
3.6.4 步骤四：验证证明	10
3.6.5 进阶代码说明	10
4 零知识证明-可靠性探究	11

5 实验总结 & 心得体会

12

1 实验要求

参考教材实验 3.1, 假设 Alice 希望证明自己知道如下方程的解 $x^3 + x + 5 = out$, 其中 out 是大家都知道的一个数, 这里假设 out 为 35, 而 $x = 3$ 就是方程的解请实现代码完成证明生成和证明的验证。

2 实验原理

2.1 实验背景

在传统认证或秘密验证中, 证明者往往需要直接提交密码、密钥或者答案本身, 验证者才能确认其是否掌握秘密。这种方式虽然简单, 但也存在明显风险: 一旦秘密在传输或存储过程中泄露, 就会造成后续安全问题。

零知识证明 (Zero-Knowledge Proof, ZKP) 的目标是让证明者能够向验证者证明“自己确实知道某个秘密”, 同时又不泄露这个秘密本身 [2]。对于本实验而言, Alice 希望向 Bob 证明自己知道方程

$$x^3 + x + 5 = 35$$

的一个解, 但又不希望把这个解直接告诉 Bob (我知道答案但就是不告诉你)。题目中给出的已知正确解是 $x = 3$, 因此实验任务可以概括为: 在不公开 x 的前提下完成证明生成与证明验证。

2.2 零知识证明的基本性质

一个合格的零知识证明系统通常需要满足以下三个性质:

1. **完备性 (Completeness)**: 若证明者确实知道秘密, 并且诚实执行协议, 则验证者应当接受证明。
2. **可靠性 (Soundness)**: 若证明者并不知道秘密, 则不应轻易伪造出可通过验证的证明。
3. **零知识性 (Zero-Knowledge)**: 验证者在验证成功后, 只知道“证明者掌握该秘密”, 而不能从证明过程中恢复出秘密本身。

因此, 本实验的目标不是简单计算出 $x = 3$, 而是设计一个协议, 使得 Bob 能够确认 Alice 知道解, 但又不能直接得到解。

2.3 从承诺开始理解证明原理

在零知识证明中, **承诺 (Commitment)** 是非常核心的思想。可以将其理解为: 证明者先给出一个与秘密相关、但又不会直接泄露秘密的中间值, 从而把自己当前的“说法”先固定下来。后续验证者再基于这个承诺提出挑战, 证明者据此给出响应。这样就形成了零知识证明中经典的“承诺-挑战-响应”结构。

本实验采用 **Schnorr 型知识证明协议** [3] 来模拟这一过程。设公开参数为素数 p 、子群阶 q 和生成元 g ，令 Alice 掌握的秘密 witness 为 x ，对应的公开值为：

$$pk = g^x \bmod p.$$

其中 pk 是公开陈述， x 是秘密 witness。Bob 可以知道 pk ，但不能从中直接恢复出 x 。

在交互式 Schnorr 协议中，证明过程可分为以下四步：

1. **承诺阶段**：Alice 随机选择一次性随机数 $r \in \mathbb{Z}_q$ ，计算

$$R = g^r \bmod p.$$

这里的 R 就是承诺值。由于 r 是随机的，因此 R 不会直接泄露秘密 x 。

2. **挑战阶段**：Bob 向 Alice 发送一个挑战值 c ，要求 Alice 针对该挑战证明自己确实知道秘密。
3. **响应阶段**：Alice 根据随机数 r 和秘密 x 计算

$$z = r + c \cdot x \bmod q.$$

并把响应值 z 发送给 Bob。

4. **验证阶段**：Bob 检查如下等式是否成立：

$$g^z \stackrel{?}{=} R \cdot pk^c \pmod{p}.$$

上述等式成立的原因是：

$$g^z = g^{r+cx} = g^r \cdot g^{cx} = R \cdot (g^x)^c = R \cdot pk^c \pmod{p}.$$

因此，只要 Alice 确实知道秘密 x ，她就能够针对挑战构造出满足等式的响应；而不知道 x 的人则难以伪造通过验证的证明。

2.4 非交互化处理

标准 Schnorr 协议是交互式协议，需要 Alice 和 Bob 来回通信。为了便于程序实现，本实验采用 **Fiat-Shamir 变换** [1]，将原本由 Bob 给出的挑战改为通过哈希函数自动生成：

$$c = H(out, pk, R) \bmod q.$$

这样一来，Alice 可以在本地一次性完成承诺、挑战和响应的计算，最终输出证明

$$(out, pk, R, z).$$

Bob 拿到证明后，只需重新计算挑战值，再检查验证等式是否成立即可。这就把原本的交互式证明改造成了非交互证明。

2.5 进阶实验原理：基于 libsnark 的通用零知识证明

如果希望把本实验进一步升级成更严格的通用零知识证明，而不仅仅是教学化的 Schnorr 演示，那么可以使用 libsnark 库来完成基于 R1CS 的 zkSNARK 证明。此时证明的对象不再只是“我知道一个秘密数”，而是“我知道一个私密输入 x ，它满足公开方程 $x^3 + x + 5 = 35$ ”。

2.5.1 约束系统的构造

为了让 libsnark 处理该方程，可以先引入两个中间变量：

$$t_1 = x \cdot x, \quad t_2 = t_1 \cdot x.$$

于是原方程可被拆成以下三条约束：

$$\begin{cases} x \cdot x = t_1 \\ t_1 \cdot x = t_2 \\ (t_2 + x + 5) \cdot 1 = out \end{cases}$$

其中公开输入为 $out = 35$ ，私密 witness 为：

$$x = 3, \quad t_1 = 9, \quad t_2 = 27.$$

2.5.2 R1CS 与 zkSNARK 流程

libsnark 接受的核心输入之一是 R1CS (Rank-1 Constraint System, 秩一约束系统)。对本实验来说，上述三条关系都可以写成秩一约束，随后交由 zkSNARK 系统处理。整个流程通常包括：

1. 构造约束系统；
2. 生成证明密钥和验证密钥；
3. 输入公开输入与私密 witness，生成证明；
4. 验证者仅使用公开输入和验证密钥完成验证。

与前文的方法相比，这种方式证明的已经不是“知道某个秘密”，而是“知道一个满足公开约束系统的私密输入”，因此更符合通用零知识证明的标准形式 [4]。

3 实验过程

3.1 实验环境

本实验分为基础实现与进阶实现两个部分，分别采用不同的工具与库。

在基础实现中，主要使用 Python 标准库完成零知识证明流程的模拟，所使用主要库及其作用如下：

- **hashlib**：用于实现 Fiat-Shamir 变换，通过哈希函数生成挑战值；
- **secrets**：用于生成密码学安全随机数，以模拟承诺阶段中的随机性；

在进阶实现中，采用 **libsnark** 库完成 zkSNARK 的完整流程。该库能够直接处理 R1CS (Rank-1 Constraint System) 约束系统，并提供从密钥生成、证明生成到证明验证的一体化接口，极大简化了零知识证明系统的实现过程。

本实验中涉及的核心接口如表 1 所示。

表 1: libsnark 核心接口说明

接口名称	功能说明
<code>protoboard<FieldT></code>	用于构建并维护整个约束系统，包含变量定义及约束关系。
<code>pb_variable<FieldT></code>	用于定义电路中的变量，包括公开输入 (public input) 和私密见证 (witness)。
<code>r1cs_constraint<FieldT></code>	用于描述单条 R1CS 约束，即形如 $a \cdot b = c$ 的秩一约束关系。
<code>r1cs_gg_ppzksnark_generator</code>	根据约束系统生成证明密钥 (proving key) 和验证密钥 (verification key)。
<code>r1cs_gg_ppzksnark_prover</code>	在给定 witness 的情况下，生成对应的零知识证明 (proof)。
<code>r1cs_gg_ppzksnark_verifier_strong</code>	使用验证密钥与公开输入，对证明进行验证，判断其有效性。

3.2 实验参数

为了便于代码演示，本实验选用一个较小但足以说明过程的安全素数群：

$$p = 1019, \quad q = 509, \quad g = 3.$$

其中 $q \mid (p - 1)$, g 是该子群的生成元。由于本实验只用于模拟而非真实部署，为了方便本地计算，因此没有采用大整数群参数。

3.3 实验模拟步骤

3.3.1 步骤一：验证公共方程并确定 witness

先对方程

$$x^3 + x + 5 = 35$$

进行枚举检查，确认在实验采用的整数范围内它只有一个整数解：

$$x = 3.$$

这一步的作用是明确实验中的公共语句与秘密 witness。其中，方程和 $out = 35$ 是公开信息，而 $x = 3$ 只由 Alice 掌握。

3.3.2 步骤二：生成公开陈述

利用 witness 生成公开陈述：

$$pk = g^x \bmod p = 3^3 \bmod 1019 = 27.$$

在后续证明中，Alice 并不直接发送 x ，而是围绕公开值 pk 构造零知识证明。

3.3.3 步骤三：模拟承诺、挑战与响应

程序随机生成一次性随机数 r ，先计算承诺值 R ，再通过哈希函数生成挑战值 c ，最后计算响应值 z ，最终得到证明对象：

$$(out, pk, R, z).$$

由于挑战由哈希函数自动生成，因此这里实现的是非交互版本，Alice 可以一次性生成完整证明。

3.3.4 步骤四：验证证明

验证者拿到证明后，重新计算挑战值，并检查

$$g^z = R \cdot pk^c \pmod{p}$$

是否成立。若成立，则说明证明与 Alice 掌握的秘密一致；若不成立，则拒绝该证明。

3.4 核心代码说明

程序的核心由四个部分组成：

1. `equation(x)`: 计算 $x^3 + x + 5$ 。
2. `find_integer_solutions(out)`: 枚举公共方程的整数解。
3. `generate_proof(out, witness)`: 生成 Schnorr/Fiat-Shamir 证明。
4. `verify_proof(proof)`: 完成证明校验。

生成证明和证明校验的关键实现代码如下。

Listing 1: 零知识证明核心实现

```
1 import hashlib
2 import secrets
3 from dataclasses import dataclass
4
5 P = 1019
6 Q = 509
7 G = 3
8
9 def generate_proof(out: int, witness: int) -> Proof:
10     nonce = secrets.randbelow(Q - 1) + 1
11     public_key = pow(G, witness % Q, P)
12     commitment = pow(G, nonce, P)
13     challenge = hash_to_scalar(out, public_key, commitment)
14     response = (nonce + challenge * (witness % Q)) % Q
15     return Proof(out, public_key, commitment, response)
16
17 def verify_proof(proof: Proof) -> bool:
18     challenge = hash_to_scalar(proof.out, proof.public_key, proof.commitment)
19     left = pow(G, proof.response, P)
20     right = (proof.commitment * pow(proof.public_key, challenge, P)) % P
21     return left == right
```


3.5 程序运行结果

执行程序输出如下。由于随机数每次都会变化，因此承诺值与响应值会有所不同，但验证结论保持不变。

Listing 2: 程序运行输出

```
1  === 公共语句 ===
2  方程:  $x^3 + x + 5 = 35$ 
3  公开参数:  $p=1019, q=509, g=3$ 
4
5  === witness 检查 ===
6  整数范围  $[-100, 100]$  内的解: [3]
7  Alice 持有的 witness  $x = 3$ 
8  代入验证:  $3^3 + 3 + 5 = 35$ 
9
10 === 证明生成 ===
11 {
12     "out": 35,
13     "public_key": 27,
14     "commitment": 836,
15     "response": 342
16 }
17
18 === 证明验证 ===
19 验证结果: True
20
21 === 篡改测试 ===
22 篡改后的验证结果: False
```

从结果可以看出：

1. 正常生成的证明能够通过验证。
2. 只要对证明中的响应值做微小篡改，验证就会失败。
3. 这说明程序已经实现了完整的“证明生成-证明验证”闭环。

3.6 进阶实验：libsark 实现模拟过程

3.6.1 步骤一：构造 R1CS 约束系统

在进阶实现中，不再直接围绕 Schnorr 承诺构造证明，而是先把方程

$$x^3 + x + 5 = 35$$

拆成约束系统。程序中通过 `protoboard` 定义变量 `out`、`x`、`x2`、`x3`，再依次加入三条约束：

$$x \cdot x = x2, \quad x2 \cdot x = x3, \quad (x3 + x + 5) \cdot 1 = out.$$

3.6.2 步骤二：设置公开输入与私密 witness

这里取公开输入 `out=35`，私密 witness 为：

$$x = 3, \quad x2 = 9, \quad x3 = 27.$$

如果这些值满足全部约束，则说明 witness 与公开方程是一致的。

3.6.3 步骤三：生成证明密钥、验证密钥与证明

在 libsark 中，先调用 `r1cs_gg_ppzksnark_generator` 根据约束系统生成密钥对，再调用 `r1cs_gg_ppzksnark_p` 输入公开输入和私密 witness 生成证明。

3.6.4 步骤四：验证证明

验证阶段调用 `r1cs_gg_ppzksnark_verifier_strong_IC`，只输入公开输入和证明进行校验。若验证成功，则说明证明者确实知道一个满足约束系统的私密输入，而无需暴露该输入本身。

3.6.5 进阶代码说明

进阶实验核心流程如下：

Listing 3: libsark 进阶实现核心流程

```
1 typedef alt_bn128_pp ppT;
2 typedef Fr<ppT> FieldT;
3
4 ppT::init_public_params();
5
6 protoboard<FieldT> pb;
7 pb_variable<FieldT> out, x, x2, x3;
8
```

```
9 out.allocate(pb, "out");
10 x.allocate(pb, "x");
11 x2.allocate(pb, "x2");
12 x3.allocate(pb, "x3");
13 pb.set_input_sizes(1);
14
15 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x, x, x2));
16 pb.add_r1cs_constraint(r1cs_constraint<FieldT>(x2, x, x3));
17 pb.add_r1cs_constraint(
18     r1cs_constraint<FieldT>(x3 + x + FieldT("5"), FieldT::one(), out)
19 );
20
21 pb.val(out) = FieldT("35");
22 pb.val(x) = FieldT("3");
23 pb.val(x2) = FieldT("9");
24 pb.val(x3) = FieldT("27");
25
26 auto cs = pb.get_constraint_system();
27 auto keypair = r1cs_gg_ppzksnark_generator<ppT>(cs);
28 auto proof = r1cs_gg_ppzksnark_prover<ppT>(
29     keypair.pk, pb.primary_input(), pb.auxiliary_input()
30 );
31 bool ok = r1cs_gg_ppzksnark_verifier_strong_IC<ppT>(
32     keypair.vk, pb.primary_input(), proof
33 );
```

4 零知识证明-可靠性探究

实验做完了，我们来探究一下这个 0 知识证明为啥可靠呢，从上课几周的经验来看，现在的加密可靠性都依赖于巨大计算量：从协议执行角度看，Schnorr/Fiat-Shamir 证明本身的计算量并不高。证明生成阶段主要涉及两次模幂运算：一次计算承诺值 $R = g^r \bmod p$ ，一次计算公开值 $pk = g^x \bmod p$ ；验证阶段同样主要涉及两次模幂运算：一次计算 $g^z \bmod p$ ，一次计算 $pk^c \bmod p$ ，再配合一次模乘即可完成校验。因此，对诚实参与方而言，该协议的运行开销大致是常数次模幂运算，复杂度可以近似看作 $O(\log q)$ 次乘法级别。

从攻击者角度看，真正决定可靠性的，是恢复秘密 x 或伪造响应所需要付出的计算代价。在本实验参数下， $q = 509$ ，秘密指数的取值空间最多只有 509 种。也就是说，如果攻击者直接暴力枚举

$x = 0, 1, \dots, 508$, 逐个检查

$$g^x \stackrel{?}{=} pk \pmod{p}$$

是否成立, 最多只需要 509 次尝试就可以恢复秘密。对于现代计算机而言, 这个代价几乎可以忽略, 因此这里的小素数参数只能用于教学演示, 而不能提供真实安全性。

若把参数提升到实际密码系统常见的大素数规模, 例如令子群阶 q 约为 2^{256} , 那么暴力枚举秘密的复杂度将提升到

$$2^{256} \approx 1.16 \times 10^{77}$$

次尝试。这一数量级已经远远超过现实中任何计算设备能够承受的范围。即使假设一台设备每秒可以完成 10^{12} 次尝试, 穷举全部空间所需时间仍然约为

$$\frac{2^{256}}{10^{12}} \approx 1.16 \times 10^{65} \text{ 秒,}$$

折合约为

$$3.7 \times 10^{57} \text{ 年,}$$

这在实际上是不可完成的。

因此, 本方法的可靠性可以这样理解: 协议本身对诚实双方来说只需要少量模幂运算, 开销较低; 而对攻击者而言, 若参数足够大, 恢复秘密或伪造证明的代价会随着子群阶 q 的增大呈指数级上升。也正因为如此, 实验中小素数适合展示协议流程, 而真正部署时必须使用大素数参数, 才能使该方法具备实际安全性。

5 实验总结 & 心得体会

本次实验围绕“证明自己知道方程解”这一目标, 完成了一个简化的零知识证明原型。通过实验可以看出, 零知识证明并不只是一个抽象概念, 而是可以被拆解为明确的协议步骤: 先做承诺, 再生成挑战, 随后给出响应, 最后由验证方检查等式。

从实现角度看, 本实验采用了 Schnorr 协议结合 Fiat-Shamir 变换的非交互形式。这样做的好处是: 代码量小、逻辑清晰、能够完整覆盖证明生成和证明验证两个关键环节。虽然它还不是一个通用的 zkSNARK 实现, 但已经足以帮助理解零知识证明的基本思想与工程结构。

但是存在一个巨大槽点, Bob 只能知道你手里有一个答案, 怎么确实是真答案?, 所以我还尝试了进阶实验: 把本实验扩展成“对任意多项式约束做真正的零知识证明”。尝试过程中接触了新的库、新的流程, 收获满满。

参考文献

- [1] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Advances in Cryptology — CRYPTO '86*, pages 186–194. Springer, 1987.
- [2] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [3] Claus-Peter Schnorr. Efficient signature generation by smart cards. In *Advances in Cryptology — CRYPTO '89 Proceedings*, pages 239–252. Springer, 1991.
- [4] SCIPR Lab. libsnark: a c++ library for zksnark proofs. <https://github.com/scipr-lab/libsnark>, 2024.